

while-Schleife in Java

Kategorie	Beschreibung
Grundstruktur	<code>while (Bedingung) { // Anweisungen }</code>
Bedingung	Wird vor jeder Iteration geprüft; ist sie <code>false</code> , wird die Schleife beendet
Initialisierung	Findet vor der Schleife statt, z. B. <code>int i = 0;</code>
Aktualisierung	Muss innerhalb der Schleife erfolgen, z. B. <code>i++</code>
Beispiel	<code>int i = 0; while (i < 5) { log.info("wert: {}", i); i++; }</code>
Endlosschleife	<code>while (true) { // Unendliche wiederholung }</code>
Verschachtelte Schleifen	<code>while (i < 3) { int j = 0; while (j < 3) { log.info("i={}, j={}", i, j); j++; } i++; }</code>
<code>break</code> -Nutzung	Beendet die Schleife vorzeitig: <code>if (i == 5) break;</code>
<code>continue</code> -Nutzung	Überspringt die aktuelle Iteration: <code>if (i == 3) { i++; continue; }</code>
Schleife über Arrays	<code>int i = 0; while (i < array.length) { log.info!("{}", array[i]); i++; }</code>
Schleife über <code>List</code>	<code>int i = 0; while (i < list.size()) { log.info!("{}", list.get(i)); i++; }</code>
Alternative mit Iterator	<code>Iterator<String> it = list.iterator(); while (it.hasNext()) { log.info(it.next()); }</code>
Leere Schleife	<code>while (bedingung);</code> (keine Anweisungen – meist zu vermeiden)
Verwendung mit Streams	Wird meist durch Streams ersetzt: <code>list.stream().forEach(s -> log.info(s));</code>